Neural Compression for the CommaVQ Challenge

A Frame-Level Predictor with Quantization and Entropy Coding

Yuval Levental — February 2026

Abstract

This report describes a lossless compression solution for the comma.ai commaVQ challenge, achieving 2.7x compression on 5,000 minutes of driving video tokens. The approach combines a small transformer-based frame predictor (5.3M parameters), 8-bit weight quantization, and Asymmetric Numeral Systems (ANS) entropy coding. Unlike the provided 307M-parameter GPT model which is impractically large (614MB), our solution uses a model that compresses to just 4.5MB, making the total submission viable. This places 3rd on the leaderboard behind szabolcs-cs (3.4x) and BradyWynn (2.9x).

1. Introduction

1.1 The Challenge

The comma.ai compression challenge requires lossless compression of 5,000 one-minute driving video segments. The data has already been processed through a VQ-VAE, converting raw 256×128 video frames into 16×8 grids of 10-bit tokens. Each minute contains 1,200 frames, and each frame contains 128 tokens, yielding a total of 768 million tokens (960MB raw).

1.2 Why the Provided Model Fails

Comma.ai provides a pre-trained GPT-2 medium model (~307M parameters) trained on 3 million minutes of driving data. While this model produces good probability distributions for entropy coding, it has two fatal flaws:

Problem	Impact
Model size: 614MB	Larger than the entire 1st place submission (270MB)
Speed: ~130 tokens/sec	Would take 68 days to decompress all data

The submission must include decompression code and model weights. Since the model alone exceeds any reasonable compressed size, the provided GPT is essentially a trap for naive approaches.

2. Related Work

2.1 Self-Compressing Neural Networks (szabolcs-cs, 1st place)

Cséfalvay and Imber [1] introduced Self-Compression, a method that learns bit depths during training. Their key insight is that quantization parameters (scale and bit depth) can be made differentiable and optimized jointly with network weights. When a channel's bit depth reaches zero, it can be pruned entirely. This achieves floating-point accuracy with as few as 3% of the original bits. Their loss function combines task loss with a compression penalty: $\Lambda(x) = \Lambda\blacksquare(x) + \gamma Q$, where Q is the average bit depth across all weights.

2.2 Frame-Level Prediction (BradyWynn, 2nd place)

Wynn [2] made a crucial architectural observation: the original GPT predicts tokens autoregressively within each frame (128 forward passes per frame). By assuming tokens within a frame are conditionally independent given previous frames, the model can predict all 128 tokens simultaneously, providing a 128× speedup with minimal loss in prediction quality. Wynn also analyzed scaling laws to find the optimal model size, noting that larger models compress better but take more space.

2.3 Entropy Coding Foundations

Arithmetic coding and its modern variant, Asymmetric Numeral Systems (ANS) [3], achieve near-optimal compression by encoding symbols using approximately $-\log\blacksquare(p)$ bits, where p is the predicted probability. The key requirement is a probability model—the better the model predicts the data, the fewer bits needed.

3. Our Approach

3.1 Architecture Overview

Our solution has three components: (1) a frame-level transformer predictor, (2) 8-bit weight quantization with zlib compression, and (3) ANS entropy coding via the constriction library [4].

Component	Description	Size/Speed
Frame Predictor	6-layer transformer, 256 dim, 4 heads	5.3M params (20MB)
Quantization	8-bit uniform quantization + zlib	4.5MB compressed
Entropy Coder	ANS via constriction library	~339MB output

3.2 Frame Predictor Architecture

The predictor takes 8 previous frames (8 × 128 = 1024 tokens) as context and outputs probability distributions for all 128 tokens of the next frame simultaneously. Components: token embedding (1024 → 256), position embedding (128 positions), frame embedding (8 frames), transformer encoder (6 layers, 4 heads, 1024-dim FFN), and output projection (256 → 1024). Unlike autoregressive models that predict one token at a time, our model predicts an entire frame in a single forward pass, sacrificing some within-frame dependencies for massive speedups.

3.3 Training

The model was trained on 500 segments using cross-entropy loss with AdamW (lr=3e-4), cosine annealing over 30 epochs, batch size 512 across 3 GPUs. Training took ~37 hours on 3× Tesla P40, achieving final loss of 2.511 (3.62 bits/token). Theoretical compression: 10/3.62 = 2.76×, reduced to ~2.7× with model overhead.

3.4 Model Quantization

Unlike the learned bit depths of Cséfalvay and Imber [1], we use simple post-training quantization: (1) **8-bit uniform quantization**—each weight tensor scaled to [0, 255] using per-tensor min/max, and (2) **zlib compression** at level 9. Result: 20MB → 4.5MB (4.75× model compression). Prediction quality remains high (91% argmax match with original). While simpler to implement, this approach likely leaves compression gains on the table compared to learned quantization, which can allocate more bits to important weights and prune unnecessary channels entirely.

3.5 Entropy Coding with ANS

For each frame: (1) run predictor on previous 8 frames to get probability distributions, (2) use constriction's ANS coder with categorical distributions to encode actual tokens, (3) store first 8 frames raw (no context available). ANS achieves compression close to entropy while being faster than traditional arithmetic coding.

4. Results

Metric	Value	Metric	Value
Raw data	960 MB	Compression time	~12 hours
Compressed	353 MB	Decompression time	~10 hours
Ratio	2.7× (official)	Model in submission	4.5 MB

4.1 Leaderboard Comparison

Rank	Submission	Ratio	Approach
1st	szabolcs-cs	3.4×	Self-compressing neural networks
2nd	BradyWynn	2.9×	Frame-level prediction, scaling laws
3rd	This work	2.7×	Frame predictor + 8-bit quant + ANS

5. Discussion

5.1 Key Insights

The model size trap: The provided 614MB GPT is a deliberate obstacle. Any successful solution must train a smaller model or use self-compression techniques. **Frame-level prediction:** Predicting entire frames sacrifices some quality but provides 128× speedup. **Quantization is essential:** Post-training 8-bit quantization provides 4.75× model compression with minimal quality loss.

5.2 Potential Improvements

Improvement	Potential Gain	Complexity
Train on all 5000 segments	+0.1-0.2×	Low
16-frame context	+0.1×	Low
Larger model (10-20M)	+0.1-0.2×	Medium
6-bit quantization	+0.05×	Low
Self-compression [1]	+0.3-0.5×	High

The most promising path to matching 1st place is implementing self-compression [1], which learns bit depths during training rather than applying uniform quantization afterward. This allows the network to allocate bits where they matter most and prune entire channels when their bit depth reaches zero, potentially achieving much smaller model sizes with comparable prediction quality.

6. Conclusion

We presented a 2.7× lossless compression solution using a frame-level transformer predictor, 8-bit quantization, and ANS entropy coding. While not matching szabolcs-cs's 3.4×, our solution demonstrates that competitive results are achievable with straightforward techniques. The key insight is that model size must be part of the optimization—the 614MB GPT, despite its quality, is unsuitable. Code: https://github.com/yuvallevental/commavq-compression

References

[1] Cséfalvay, S. and Imber, J. (2023). Self-Compressing Neural Networks. arXiv:2301.13142.

[2] Wynn, B. (2025). Arithmetic Coding with GPT. https://bradywynn.github.io/comma/

[3] Duda, J. (2009). Asymmetric Numeral Systems. arXiv:0902.0271.

[4] Bamler, R. constriction: Entropy Coding Primitives. https://github.com/bamler-lab/constriction

[5] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017.

[6] comma.ai. commaVQ Dataset. https://github.com/commaai/commavq

[7] Loshchilov, I. and Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR 2019.

[8] van den Oord, A. et al. (2017). Neural Discrete Representation Learning. NeurIPS 2017.